ПЕРЕДАЧА ПРОЕКТА

Детектор ИИ-кода в мердж-реквестах

Оценивает, написал ли MR человек или Claude Code, и показывает процент и значимые признаки. Языки: **C#** и **фронтенд** (JavaScript / TypeScript, React, Angular, Vue). Код анализируется только локально и никуда не отправляется.

РЕПОЗИТОРИЙ
github.com/vvvq3/DetectAICodeTest

Ветка по умолчанию — `claude/inspiring-lamport-i2ixa7`

1 Запуск

```
git clone https://github.com/vvvq3/DetectAICodeTest.git
cd DetectAICodeTest
bash run.sh web            # http://localhost:5057 — .NET 10 ставится и проект собирается сам
```

Windows: `run.cmd web`. Публичная ссылка с паролем: `bash share.sh git.dextechnology.com`.

2 Сборка и тесты

```
dotnet build src/AiDetect.Cli -c Release -o .build
dotnet test tests/AiDetect.Tests              # C# и ядро
dotnet test tests/AiDetect.Frontend.Tests     # фронтенд
```

3 Что где лежит

`src/AiDetect.Core`	Ядро, общее для всех языков: загрузка MR из GitLab, git и JSON, диффы, сигналы процесса и формы MR, модель и итоговый процент, калибровка, отчёты. Интерфейс языка — `Languages/ILanguageAnalyzer.cs`
`src/AiDetect.Languages.CSharp`	Признаки C# на Roslyn, классификатор файлов, конвенции проекта, стек из `.csproj`
`src/AiDetect.Languages.Frontend`	То же для JS/TS. Свой мини-парсер в `Parsing/` (JS/TS/JSX, Vue, шаблоны Angular, CSS), стек из `package.json`
`src/AiDetect.Cli`	Программа `aidetect` и веб-сервер. Список языков — `AppLanguages.cs`
`model/`	Веса моделей по языкам: `weights.csharp.json`, `weights.frontend.json`
`corpus/<язык>/`	Манифесты корпуса для оценки. Сами MR в git не хранятся, восстанавливаются: `python3 scripts/rebuild_corpus.py --lang frontend`
`scripts/`	Оценка качества (`evaluate.sh csharp|frontend`), сборка корпуса, примеры отчётов, запуск на Маке `rundetectai.sh`
`tests/`	Модульные тесты (xUnit) и проверка веб-страницы (Playwright)

4 Что читать

`docs/CONTEXT.md` — начать с него
Вся история проекта: требования, архитектура, алгоритмы, решения и подводные камни.

`README.md`
Запуск, работа с GitLab, качество, настройка под свою команду.

Исследования
`docs/RESEARCH.md` (C#), `docs/frontend/RESEARCH.md` — какие признаки выдают ИИ-код.

Качество
`docs/EVALUATION.md`, `docs/frontend/EVALUATION.md` — корпус и метрики.

`CHANGELOG.md`
Версии: 1.0 — C#, 2.0 — C# и фронтенд.

`CLAUDE.md`
Правила для Claude Code в этом репозитории.

5 Как продолжать

- **Ветки.** Работать в своей ветке и вливать в основную `claude/inspiring-lamport-i2ixa7`.
- **Точки отката.** Ветки `v1` (только C#) и `v2` (C# и фронтенд), теги `v1.0.0` и `v2.0.0` не менять.
- **Новый язык** — отдельный проект `src/AiDetect.Languages.<Язык>` с классом, реализующим `ILanguageAnalyzer`, и одна строка в `AppLanguages.cs`. Затем корпус в `corpus/<язык>/`, калибровка весов и тесты. Пошаговый рецепт — `docs/CONTEXT.md`, раздел 10.
- **Регрессия.** Изменения в ядре не должны менять результат существующих языков: прогнать `scripts/evaluate.sh csharp` и `scripts/evaluate.sh frontend` до и после.
- **Точность на своём коде.** Разметить свои MR (человек / ИИ) и перекалибровать веса — README, «Настройка под вашу команду».

Важно
Код анализируемых MR никуда не отправляется — никаких внешних API и ИИ-сервисов, только локальный анализ. Это правило сохраняется при любых доработках.
